



POSIX. Процессы

Процесс – адресное пространство + потоки управления + системные ресурсы

Идентификатор процесса – положительное целое число, однозначно идентифицирующее процесс в течение времени его жизни.

Время жизни процесса – это период от его создания до возврата идентификатора операционной системе.

Создается функцией **fork()** - активный: минимум один поток управления и адресное пространство.

Переход в неактивное состояние — возврат части ресурсов. Когда по отношению к неактивному процессу выполняют функцию семейства `wait()`, системе возвращаются остальные ресурсы. Последний из них – идентификатор процесса, и на этом время жизни процесса заканчивается.

Зомби-процесс – завершившийся процесс, подлежащий ликвидации после того, как код его завершения будет передан ожидающему этого другому процессу.

Процесс, создавший данный, называется **родительским**, в течение времени жизни которого существует идентификатор родительского процесса. По завершении времени жизни указанного процесса родительским становится определяемый реализацией системный процесс.



POSIX. Процессы

Группа – совокупность процессов, допускающая согласованную доставку сигналов. У каждой группы имеется уникальный идентификатор. В такой роли выступает идентификатор процесса, именуемого лидером группы.

Время жизни группы - период от создания группы до момента, когда ее покидает последний процесс

Задание – набор процессов, составляющих конвейер, а также порожденных ими процессов, входящих в одну группу

Сеанс - называется множество групп процессов, сформированное для целей управления заданиями. Каждая группа принадлежит некоторому сеансу. Вновь созданный процесс присоединяется к сеансу своего создателя, в дальнейшем принадлежность сеансу может быть изменена.

Время жизни сеанса - период от создания сеанса до истечения времени жизни всех групп процессов, принадлежавших сеансу.

Лидер сеанса – процесс, создавший данный сеанс.



POSIX. Процессы

Управляющий терминал - ассоциированный с сеансом

Управляющий процесс — это лидер сеанса, установивший соединение с управляющим терминалом

С каждым процессом ассоциируется идентификатор создавшего его пользователя. Этот атрибут называется **реальным идентификатором пользователя** процесса

В момент создания процесса пользователь входил в некоторую группу, идентификатор которой называется **реальным идентификатором группы** процесса.

Для определения прав процесса (в том числе прав доступа к файлам) применяются **действующие идентификаторы пользователя и группы**, которые в общем случае могут отличаться от реальных.

Биты переустановки действующих идентификаторов пользователя (ПДИП) и группы (ПДИГ). Если эти биты взведены, то при запуске программы на выполнение действующие идентификаторы пользователя и группы процесса могут наследоваться у файла.



POSIX. Процессы

```
romanow@romanow-home ~  
$ ps
```

PID	PPID	PGID	WINPID	TTY	UID	STIME	COMMAND
1176	6692	1176	1084	pty0	197615	23:55:22	/usr/bin/bash
5892	1176	5892	7992	pty0	197615	01:11:10	/usr/bin/ps
6692	1	6692	6692	?	197615	23:55:22	/usr/bin/mintty

PID — идентификатор процесса

PPID — идентификатор родителя

PGID — идентификатор группы

TTY — управляющий терминал

UID — идентификатор пользователя



POSIX. Процессы

```
#include <unistd.h>
#include <sys/types.h>
#include <stdio.h>
int main (void) {
    pid_t ppid;
    pid_t pgid;
    /* Отменим буферизацию стандартного вывода */
    setbuf (stdout, NULL);
    printf ("Атрибуты текущего процесса: pid: %d, ppid: %d, pgid: %d\n",
           getpid (), ppid = getppid (), pgid = getpgrp ());
    /* Выделимся в новую группу */
    if (setpgid (0, 0) != 0) { perror ("setpgid (0, 0)"); }
    printf ("Новая группа текущего процесса: %d\n",
           getpgrp ());
    /* Попробуем создать новый сеанс */
    if (setsid () == (pid_t) (-1)) { perror ("setsid от имени лидера группы"); }
    /* Вернемся в прежнюю группу */
    if (setpgid (0, pgid) != 0) {
        perror ("setpgid (0, pgid)");
    }
    printf ("Группа текущего процесса после повторной смены: %d\n", getpgrp ());
    /* Повторим попытку создать новый сеанс */
    if (setsid () == (pid_t) (-1)) { perror ("setsid от имени не-лидера группы"); }
    printf ("Группа текущего процесса после создания нового сеанса: %d\n", getpgrp ());
    /* Попробуем сменить группу родительского процесса */
    if (setpgid (ppid, 0) != 0) { perror ("setpgid (ppid, 0)"); }
    /* Попробуем установить несуществующий идентификатор группы процессов */
    if (setpgid (0, ppid) != 0) { perror ("setpgid (0, ppid)"); }
    return (0);
}
```

Атрибуты текущего процесса: pid: 6868, ppid: 2880, pgid: 6868

Новая группа текущего процесса: 6868

setsid от имени лидера группы: Operation not permitted

Группа текущего процесса после повторной смены: 6868

setsid от имени не-лидера группы: Operation not permitted

Группа текущего процесса после создания нового сеанса: 6868



POSIX. Процессы

```
#include <unistd.h>
#include <sys/types.h>
#include <stdio.h>
#include <stdlib.h>

int main (void) {
    uid_t uid;
    int nsupgroups;
    gid_t *supgroupslst;
    int i;
    setbuf (stdout, NULL);
    printf ("Идентификаторы пользователя текущего процесса:\n" " реальный: %d, действующий: %d\n",
           uid = getuid (), geteuid ());
    printf ("Идентификаторы группы текущего процесса:\n" " реальный: %d, действующий: %d\n",
           getgid (), getegid ());
    /* Попробуем переустановить идентификатор пользователя процесса */
    if (setuid ((uid_t) 1) != 0) { perror ("setuid (1)"); }
    printf ("Идентификаторы пользователя текущего процесса после первой смены:\n"
           " реальный: %d, действующий: %d\n", getuid (), geteuid ());
    /* Попробуем вернуть прежний идентификатор пользователя процесса */
    if (setuid (uid) != 0) { perror ("setuid (uid)"); }
    printf ("Идентификаторы пользователя текущего процесса после второй смены:\n"
           " реальный: %d, действующий: %d\n", getuid (), geteuid ());
    /* Попробуем сменить действующий идентификатор с помощью функции seteuid() */
    if (seteuid ((uid_t) 1) != 0) { perror ("seteuid (1)"); }
    printf ("Идентификаторы пользователя текущего процесса после третьей смены:\n"
           " реальный: %d, действующий: %d\n", getuid (), geteuid ());
    return (0);
}
```

Идентификаторы пользователя текущего процесса:

реальный: 197615, действующий: 197615

Идентификаторы группы текущего процесса:

реальный: 197121, действующий: 197121

setuid (1): Operation not permitted

Идентификаторы пользователя текущего процесса после первой смены:

реальный: 197615, действующий: 197615

Идентификаторы пользователя текущего процесса после второй смены:

реальный: 197615, действующий: 197615

seteuid (1): Operation not permitted

Идентификаторы пользователя текущего процесса после третьей смены:

реальный: 197615, действующий: 197615



POSIX. Процессы

```
#include <unistd.h>
pid_t fork (void);
```

Порожденный процесс является точной копией процесса, вызвавшего `fork()` (родительского), за исключением следующих моментов:

- у порожденного процесса свой идентификатор, равно как и идентификатор родительского процесса
- у порожденного процесса собственная копия файловых дескрипторов, ссылающихся на те же описания открытых файлов, что и соответствующие дескрипторы родительского процесса
- порожденный процесс не наследует блокировки файлов, установленные родительским процессом
- порожденный процесс создается с одним потоком управления – копией того, что вызвал `fork()`

В случае успешного завершения функция `fork()` возвращает порожденному процессу 0, а родительскому процессу – идентификатор порожденного процесса. После этого оба процесса начинают независимо выполнять инструкции, расположенные за обращением к `fork()`. При неудаче родительскому процессу возвращается -1, новый процесс не создается.



POSIX. Процессы

Следующие атрибуты процесса остаются неизменными:

- идентификатор процесса;
- идентификатор родительского процесса;
- идентификатор группы процессов;
- членство в сеансе;
- реальные идентификаторы пользователя и группы процесса;
- идентификаторы дополнительных групп;
- текущий и корневой каталоги;
- маска режима создания файлов;
- атрибуты, связанные с обработкой сигналов

Переустановка идентификатора пользователя и группы по id файла, если в нем биты переустановки

```
#include <sys/wait.h>
pid_t wait (int *stat_loc);
pid_t waitpid (pid_t pid, int *stat_loc,
    int options);
```

```
#include <unistd.h>
extern char **environ;
int execl (const char *path, const char *arg0, ...
    /*, (char *) 0 */);
int execv (const char *path, char *const argv []);
int execl (const char *path, const char *arg0,
    ... /*, (char *) 0, char *const envp [] */);
int execve (const char *path, char *const argv [],
    char *const envp []);
int execlp (const char *file, const char *arg0,
    ... /*, (char *) 0 */);
int execvp (const char *file, char *const argv []);
```




POSIX. Процессы

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>
void atefunc (void) {
    printf ("Ситуация перед завершением родительского процесса\n");
    system ("ps");
}
int main (void) {
    int pid;
    int stat;
    setbuf (stdout, NULL);
    /* Зарегистрируем обработчик завершения процесса */
    if (atexit (atefunc) != 0) { perror ("ATEXIT"); exit (1); }
    /* Создадим новый процесс */
    if ((pid = fork ()) < 0) { perror ("FORK"); exit (2);}
    else if (pid == 0) {
        /* Порожденный процесс - Выполним служебную программу ps */
        printf ("Ситуация с точки зрения порожденного процесса\n");
        execl ("/bin/ps", "ps", (char *) 0);
        perror ("EXEC");
        exit (3); /* execl() завершился неудачей */
    } else {
        /* Родительский процесс */
        sleep (1);
        /* Вероятно, порожденный процесс уже завершился */
        /* Посмотрим на текущую ситуацию */
        printf ("Ситуация перед вызовом waitpid() в родительском процессе\n");
        system ("ps");
        waitpid (pid, &stat, 0);
        printf ("Статус завершения порожденного процесса с идентификатором %d:",
            pid);
        return 0;
    }
}
```



POSIX. Процессы

Ситуация с точки зрения порожденного процесса

	PID	PPID	PGID	WINPID	TTY	UID	STIME	COMMAND
I	1176	6692	1176	1084	pty0	197615	23:55:22	/usr/bin/bash
	7944	3080	3080	4648	pty1	197615	01:45:15	/usr/bin/ps
	3352	1	3352	3352	?	197615	01:16:37	/usr/bin/bash
	6692	1	6692	6692	?	197615	23:55:22	/usr/bin/mintty
	6764	1	6764	6764	?	197615	01:45:15	/cygdrive/c/Users/ROMANO~1.ROM/AppData/Local/Temp/dli
	3080	6764	3080	5264	pty1	197615	01:45:15	/cygdrive/d/Temp/cpp1/dist/Debug/Cygwin-Windows/cpp1

Ситуация перед вызовом waitpid() в родительском процессе

	PID	PPID	PGID	WINPID	TTY	UID	STIME	COMMAND
I	1176	6692	1176	1084	pty0	197615	23:55:22	/usr/bin/bash
	7944	3080	3080	4648	pty1	197615	01:45:15	/usr/bin/ps <defunct>
	6396	3080	3080	6660	pty1	197615	01:45:16	/usr/bin/ps
	6692	1	6692	6692	?	197615	23:55:22	/usr/bin/mintty
	3352	1	3352	3352	?	197615	01:16:37	/usr/bin/bash
	6764	1	6764	6764	?	197615	01:45:15	/cygdrive/c/Users/ROMANO~1.ROM/AppData/Local/Temp/dli
	3080	6764	3080	5264	pty1	197615	01:45:15	/cygdrive/d/Temp/cpp1/dist/Debug/Cygwin-Windows/cpp1

Статус завершения порожденного процесса с идентификатором 7944: 0

Ситуация перед завершением родительского процесса

	PID	PPID	PGID	WINPID	TTY	UID	STIME	COMMAND
I	1176	6692	1176	1084	pty0	197615	23:55:22	/usr/bin/bash
	1380	3080	3080	8088	pty1	197615	01:45:16	/usr/bin/ps
	3352	1	3352	3352	?	197615	01:16:37	/usr/bin/bash
	6692	1	6692	6692	?	197615	23:55:22	/usr/bin/mintty
	6764	1	6764	6764	?	197615	01:45:15	/cygdrive/c/Users/ROMANO~1.ROM/AppData/Local/Temp/dli
	3080	6764	3080	5264	pty1	197615	01:45:15	/cygdrive/d/Temp/cpp1/dist/Debug/Cygwin-Windows/cpp1



```
int main(int argc, char *argv[]) {
    int res;
    pthread_t a_thread;
    void *thread_result;
    int num=1;
    int num2=2;
    pthread_mutex_init(&mutex, NULL);
    res = pthread_create(&a_thread, NULL, thread, (void*)&num);
    if (res != 0) { perror("error 1"); exit(EXIT_FAILURE); }
    res = pthread_create(&a_thread, NULL, thread, (void*)&num2);
    if (res != 0) { perror("error 2"); exit(EXIT_FAILURE); }
    sleep(10);
    down=1;
    sleep(5);
    pthread_mutex_destroy(&mutex);
    exit(EXIT_SUCCESS);
}
```

```
int down=0;
pthread_mutex_t mutex;
void *thread(void *arg) {
    int num=(int*)arg;
    printf("thread %d started\n", num);
    while(down==0){
        pthread_mutex_lock(&mutex);
        printf("%d a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a-a\n", num);
        printf("%d b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b-b\n", num);
        pthread_mutex_unlock(&mutex);
    }
    printf("thread %d finished\n", num);
    pthread_exit((void*)"!!!!!!!!!!!!!!!!!!!!!!!!!!!!");
}
```

1	b-b
2	b-b
1	a-a
2	a-a
1	b-b
2	b-b

Без синхронизации. Блокируемый вывод



POSIX. Потоки. Синхронизация

По отношению к потокам управления вводится понятие безопасных функций, которые можно вызывать параллельно в нескольких потоках без нарушения корректности их функционирования. К числу безопасных принадлежат "чистые" функции, а также функции, обеспечивающие взаимное исключение перед доступом к разделяемым объектам. Если в стандарте явно не оговорено противное, функция считается потоковобезопасной.



POSIX. Процессы. Взаимодействие

Каналы

Сигналы

POSIX-2001, X/Open-расширение системного интерфейса " (XSI):

Очереди сообщений

Семафоры

Разделяемые сегменты памяти



Сигналы

Под сигналом понимается механизм, с помощью которого процесс или поток управления уведомляют о некотором событии, произошедшем в системе, или подвергают воздействию этого события. Примерами подобных событий могут служить аппаратные исключительные ситуации и специфические действия процессов. Термин "сигнал" используется также для обозначения самого события

- Сигналы нумеруются, номер = тип сигнала
- Сигналы посылаются процессу или отдельному потоку
- Поток имеет маску сигналов
- Действия потока (процесса) по сигналу
 - Игнорировать
 - Выполнить действие, соответствующее сигналу
 - Вызывать функцию обработки (указатель на функцию)

Функция называется асинхронно-сигнально-безопасной (АСБ), если ее можно вызывать без каких-либо ограничений при обработке сигналов. В стандарте POSIX-2001 имеется список функций, которые должны быть либо повторно входимыми, либо непрерываемыми сигналами, что превращает их в АСБ-функции.



Сигналы

Послать сигнал процессу

```
#include <signal.h>
int kill (pid_t pid, int sig);
```

Послать сигнал внутри процесса
(себе)

```
#include <signal.h>
int raise (int sig);
```

Опросить и изменить способ
обработки сигналов

```
#include <signal.h>
int sigaction (int sig, const struct sigaction
               *restrict act, struct sigaction
               *restrict oact);
```

Работа с наборами сигналов

```
#include <signal.h>
int sigemptyset (sigset_t *set);
int sigfillset (sigset_t *set);
int sigaddset (sigset_t *set, int signo);
int sigdelset (sigset_t *set, int signo);
int sigismember (const sigset_t *set,
                 int signo);
```



Сигналы

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>

void fufu(int signo){
    printf("!!!!!!%d\n",signo);
}

int main (void) {
    struct sigaction sact;
    sigset_t sset;
    int MYSIG=SIGALRM;
    (void) fflush (NULL);
    /* Снимем блокировку сигнала SIGABRT */
    if ((sigemptyset (&sset) == 0) && (sigaddset (&sset, MYSIG) == 0)) {
        sigprocmask (SIG_UNBLOCK, &sset, (sigset_t *) NULL);
    }
    /* Установим реакцию на сигнал */
    sact.sa_handler = fufu;
    sigfillset (&sact.sa_mask);
    sact.sa_flags = 0;
    sigaction (MYSIG, &sact, NULL);
    /* Пошлем себе сигнал */
    raise (MYSIG);
    sleep(5);
    return 0;
}
```

!!!!!!14

ВЫПОЛНЕНИЕ SUCCESSFUL (общее время: 5s)



Каналы

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>

int main (void) {
    int fd [2];
    int A[]={1,2,3,4,5,6,7,8,9,10};
    int B[10];
    /* Создадим безымянный канал */
    if (pipe (fd) < 0) { perror ("PIPE"); exit (1); }
    switch (fork ()) {
        case -1: perror ("FORK"); exit (2);
        case 0:
            close (fd [1]);
            if (read (fd [0], B, sizeof(int)*10) == sizeof(int)*10) {
                for(int i=0;i<10;i++)
                    printf("%d ",B[i]);
            }
            exit (0);
        }
    close (fd [0]);
    if (write (fd [1], A, sizeof(int)*10) != sizeof(int)*10) {
        perror ("WRITE TO PIPE-1");
    }
    close (fd [1]);
    wait (NULL);
    return (0);
}
```

1 2 3 4 5 6 7 8 9 10

ВЫПОЛНЕНИЕ SUCCESSFUL (общее время: 346ms)

`int pipe(FILE fd[2])` – заполняет массив `handl`-ами двух потоков данных: при записи в один из них данные становятся доступными для чтения в другом



Семафоры.

Разделяемые сегменты памяти

```
#include <unistd.h>
#include <stdio.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <sys/wait.h>

int main8 (void) {
    int size=100000;
    int *shm_ptr=new int[size];          // Зачем создавать, когда получаем
    struct sembuf P = {0, -1, 0};
    struct sembuf V = {0, 1, 0};
    int shmid;
    int semid;
    sembuf p;
    //--- получить id разделяемой памяти
    shmid = shmget (IPC_PRIVATE, size*sizeof(int), 0777);
    //--- получить id семафора
    semid = semget (IPC_PRIVATE, 1, 0777);
    //--- Установка семафора
    semctl (semid, 0, SETVAL, 1);
    switch (fork ()) {
        case -1: perror ("FORK"); return (1);
        case 0:
            //--- получить адрес разделяемой области и проверить его на -1
            if ((int) (shm_ptr = (int*) shmat (shmid, NULL, 0)) == (-1)) {
                perror ("CHILD-SHMAT"); return (2);
            }
            if (semop (semid, &p, 1) != 0) { perror ("CHILD-SEMOP-P"); return (3); }
            printf ("Процесс-потомок вошел в критический интервал\n");
            shm_ptr[0] = getpid ();          // Монопольный доступ - запись id процесса
            printf ("Процесс-потомок перед выходом из критического интервала\n");
            if (semop (semid, &V, 1) != 0) { perror ("CHILD-SEMOP-V"); return (4); }
            shmdt (shm_ptr);
            return 0;
        }
    }
```



Разделяемые сегменты памяти

```
if ((int) (shm_ptr = (int*) shmat (shmid, NULL, 0)) == (-1)) {
    perror ("PARENT-SHMAT"); return (2);
}
if (semop (semid, &p, 1) != 0) { perror ("PARENT-SEMOP-P"); return (3); }
printf ("Родительский процесс вошел в критический интервал\n");
shm_ptr[0] = getpid ();          /* Монопольный доступ */
printf ("Родительский процесс перед выходом из критического интервала\n");
if (semop (semid, &V, 1) != 0) { perror ("PARENT-SEMOP-V"); return (4); }
wait (NULL);
printf ("Идентификатор родительского процесса: %d\n", getpid ());
printf ("Идентификатор процесса в разделяемой структуре: %d\n", shm_ptr[0]);
shmdt (shm_ptr);
semctl (semid, 1, IPC_RMID);
shmctl (shmid, IPC_RMID, NULL);
return 0;
}
```